

DESENVOLVIMENTO FRONT-END II

SPRINT 3 — RELATÓRIO FINAL

Este é o documento final do trabalho. Ele acompanha a apresentação e a demo ao vivo. Registre aqui não só o que foi construído, mas o que o grupo aprendeu ao longo do semestre.

1. Identificação

Nome do Sistema	Pé na Trilha
URL do Repositório	https://github.com/DaviLLi/pe-na-trilha-frontend
URL da Demo (se houver)	Não publicado. A demonstração será executada localmente.
Integrantes	Davi Martinelli Landal; Victor Schmitz Alvim de Oliveira; Hiago Farias Veloso; Hiuri Machado Cardozo
Data da Apresentação	Conforme cronograma da disciplina

2. Checklist de Entrega

Marque o que o grupo está entregando. O professor vai verificar cada item durante a apresentação.

Item	Status
Repositório com código completo e histórico de commits de todos os integrantes	☐ Sim ☒ Não
README com instruções de como instalar as dependências e rodar o projeto (npm install + npm run dev)	☒ Sim ☐ Não
Aplicação React rodando sem erros no navegador	☒ Sim ☐ Não
Múltiplas páginas com menu de navegação funcionando	☒ Sim ☐ Não
Consumo de API RESTful com CRUD completo (Create, Read, Update, Delete)	☒ Sim ☐ Não
Relatório Final (este documento) preenchido	☒ Sim ☐ Não
Demo ao vivo preparada para a apresentação	☒ Sim ☐ Não

3. O Que Foi Construído

3.1 Páginas e componentes entregues

#	Página / Componente	Tipo	Funciona na demo?
1	Lista de Trilhas (/trilhas)	☒ Página ☐ Componente	☒ Sim ☐ Não

2	Cadastro de Trilha (/trilhas/nova)	☒ Página ☐ Componente	☒ Sim ☐ Não
3	Editar Trilha (/trilhas/:id/editar)	☒ Página ☐ Componente	☒ Sim ☐ Não
4	Header e menu de navegação	☐ Página ☒ Componente	☒ Sim ☐ Não
5	TrilhaForm	☐ Página ☒ Componente	☒ Sim ☐ Não
6	PostTrilha	☐ Página ☒ Componente	☒ Sim ☐ Não

3.2 Integração com o Back-end (CRUD RESTful)

Entidade gerenciada no CRUD	<i>Trilhas</i>
Endpoints consumidos	<i>GET /trilhas; GET /trilhas/:id; POST /trilhas; PUT /trilhas/:id; DELETE /trilhas/:id</i>
Como fez as requisições?	<i>Axios com async/await. O cliente HTTP fica em src/services/api.js e as operações CRUD em src/services/trilhasService.js.</i>
Tratamento de erros	<i>Uso de try/catch, mensagens de erro na interface, estados de carregamento, validação dos campos no backend e middleware central para erros, IDs inválidos, JSON malformado e rotas inexistentes.</i>
Integração funcionando?	☒ Totalmente ☐ Parcialmente ☐ Não foi possível

3.3 O que ficou fora do escopo e por quê

Autenticação, cadastro de usuários, JWT, controle de acesso, upload de imagens, comentários e avaliações ficaram fora do escopo porque a Sprint 3 definiu somente a entidade Trilha, com os campos título, descrição e endereço. A publicação online também não foi realizada; a apresentação será feita localmente com MongoDB Atlas.

4. Qualidade do Código React

4.1 Decisões técnicas e de organização

Organização do projeto	<i>Frontend separado em src/components, src/pages, src/services e src/styles. Backend separado em config, controllers, middlewares, models e routes.</i>
Gerenciamento de estado	<i>useState para dados, carregamento, envio e mensagens de erro; useEffect para buscar dados da API; estado local por página.</i>
Roteamento	<i>React Router DOM com BrowserRouter, Routes, Route, NavLink, Link, Navigate, useNavigate e useParams.</i>
Estilização	<i>CSS tradicional separado por responsabilidade, preservando a</i>

	<i>identidade visual criada na Sprint 2 e adicionando estilos para formulários, edição, carregamento e erros.</i>
Boas práticas adotadas	<i>Componentes reutilizáveis, separação de responsabilidades, serviço Axios centralizado, async/await, cancelamento de requisição na listagem, validação no frontend e backend, variáveis de ambiente, tratamento centralizado de erros e conexão MongoDB isolada.</i>

5. Controle de Versão

Total de commits no projeto	<i>13 commits registrados até o momento. As alterações da Sprint 3 ainda precisam ser commitadas.</i>
Organização dos arquivos no repositório	<i>Na raiz ficam frontend, backend e README. O frontend usa src/components, src/pages, src/services e src/styles. O backend usa src/config, src/controllers, src/middlewares, src/models e src/routes.</i>
Todos os integrantes commitaram?	<i>☐ Sim ☒ Não (o histórico atual registra commits apenas de Davi Martinelli; os demais integrantes ainda não aparecem como autores.)</i>

6. Reflexão do Grupo

6.1 O que o grupo aprendeu na prática

Ao aplicar React no projeto, o grupo compreendeu melhor a diferença entre componentes e páginas, o fluxo de propriedades, formulários controlados, renderização condicional e atualização de listas sem recarregar a página. A integração com Axios também deixou mais claro como useEffect, useState e async/await trabalham juntos para buscar e alterar dados. No backend, ficou evidente a importância de separar rotas, controllers, model e conexão com o banco.

6.2 O que faria diferente

Se o projeto começasse novamente, o grupo definiria a arquitetura full stack e o contrato da API desde a primeira sprint. Também organizaria componentes, páginas e serviços desde o início, adotaria commits menores e distribuídos entre todos os integrantes e criaria testes automatizados para o CRUD. Isso reduziria a migração do localStorage e facilitaria a integração entre frontend e backend.

6.3 O projeto tem futuro?

Sim. O Pé na Trilha pode evoluir para uma plataforma mais completa. Os próximos passos seriam publicar frontend e backend, adicionar testes, melhorar acessibilidade e responsividade e, em outro escopo, implementar autenticação, perfis, fotos, comentários, avaliações, mapas e informações de dificuldade e segurança das trilhas.

Bom trabalho ao longo do semestre.